

Modern Rendering Pipelines

Computer Graphics
Instructor: Sungkil Lee

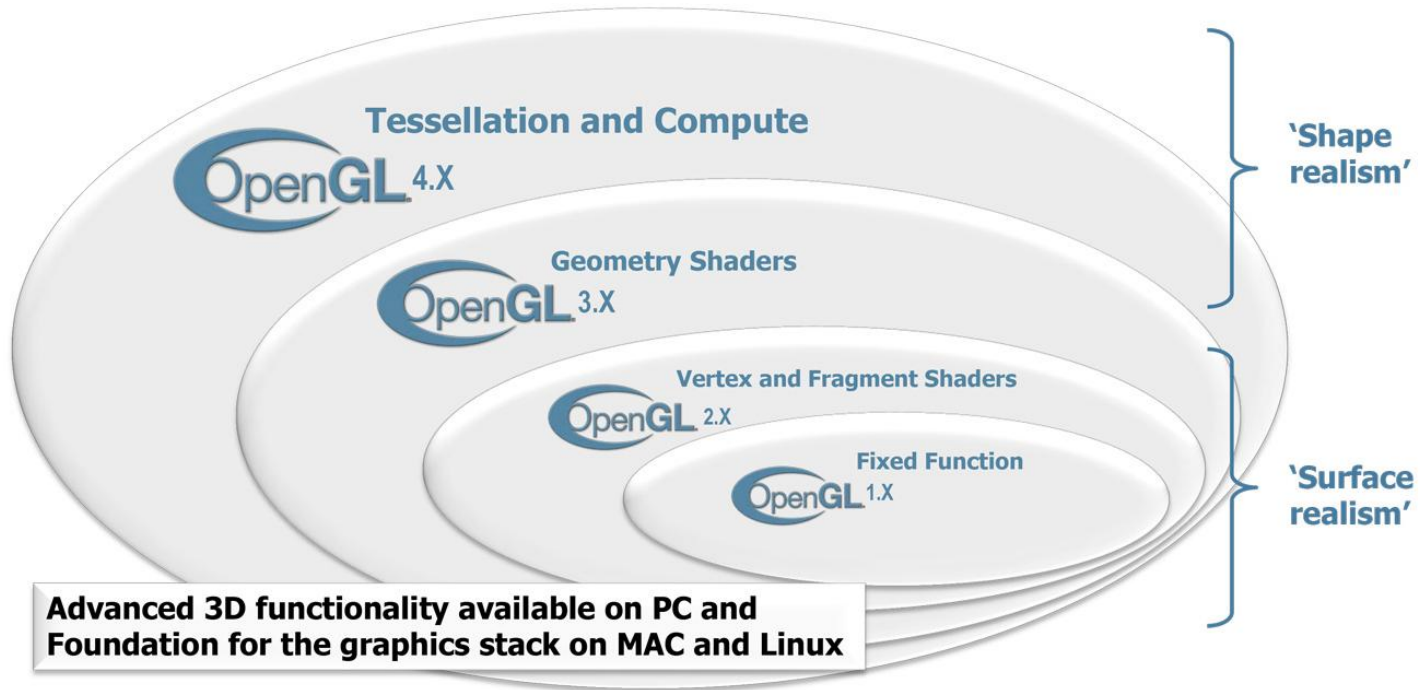
Contents

- **Modern GPU Architecture**
- **Forward Rendering Pipeline**
- **Deferred Rendering Pipeline**

Modern GPU Architecture

Evolution of OpenGL Shaders

- **OpenGL for each Hardware generation**

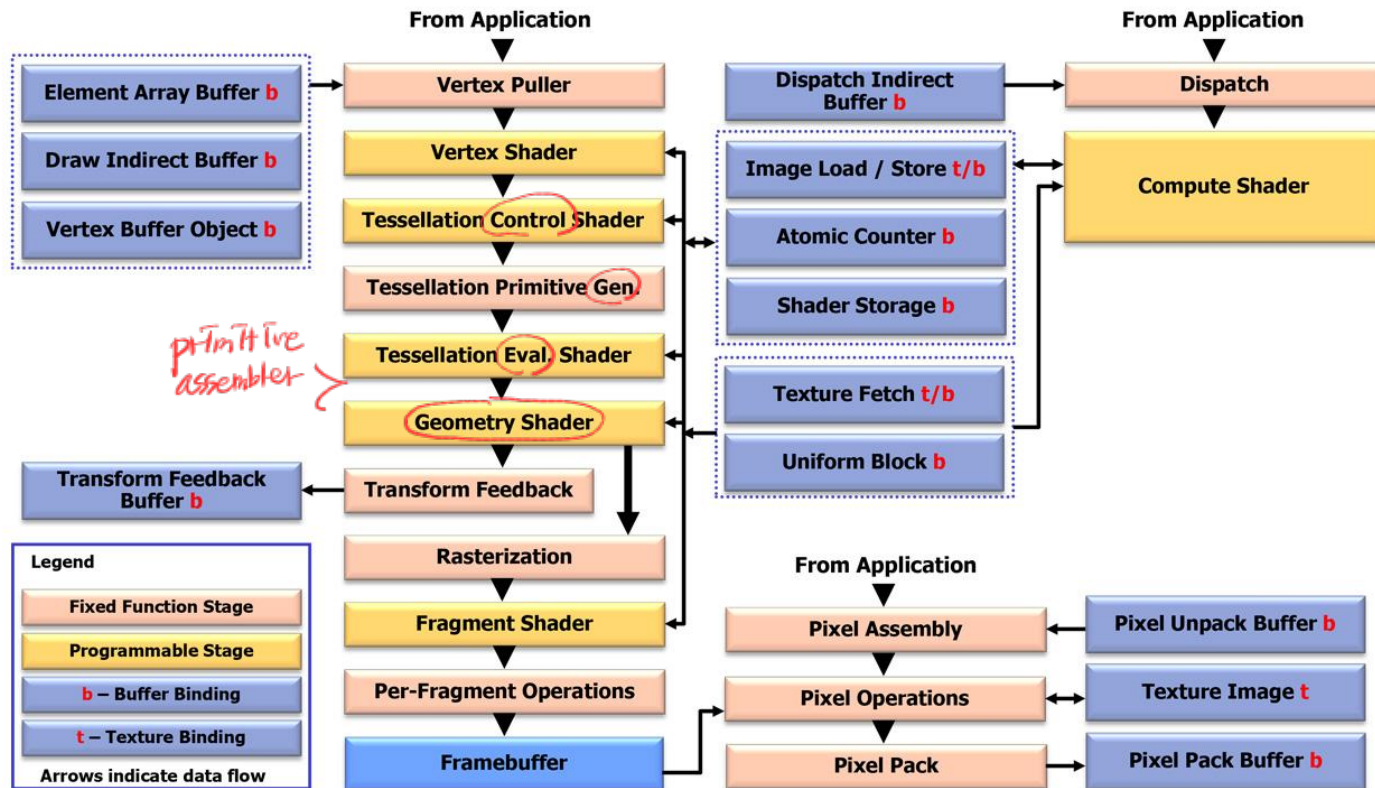


© Copyright Khronos Group 2012 | Page 23

Modern Architecture of GPU Shaders

- Shader architecture since OpenGL 4.3

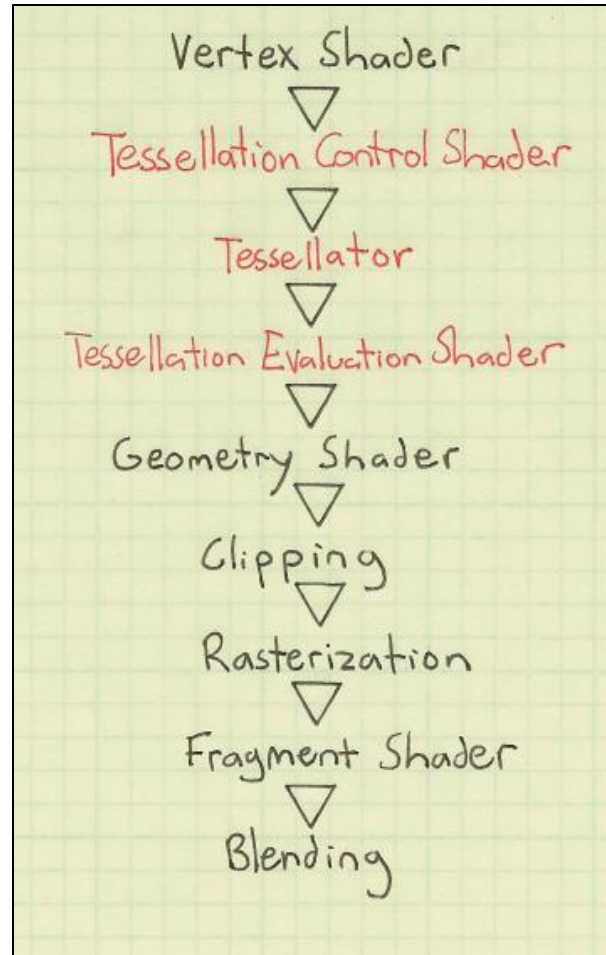
OpenGL 4.3 with Compute Shaders



© Copyright Khronos Group 2012 | Page 28

Modern Architecture of GPU Shaders

- Shader architecture of OpenGL (simplified view)



Vertex shader
Tessellation control shader
Tessellator
Tessellation Evaluation
Geometry shader
Clipping
Rasterization
Fragment shader
blending

Tessellation Shaders

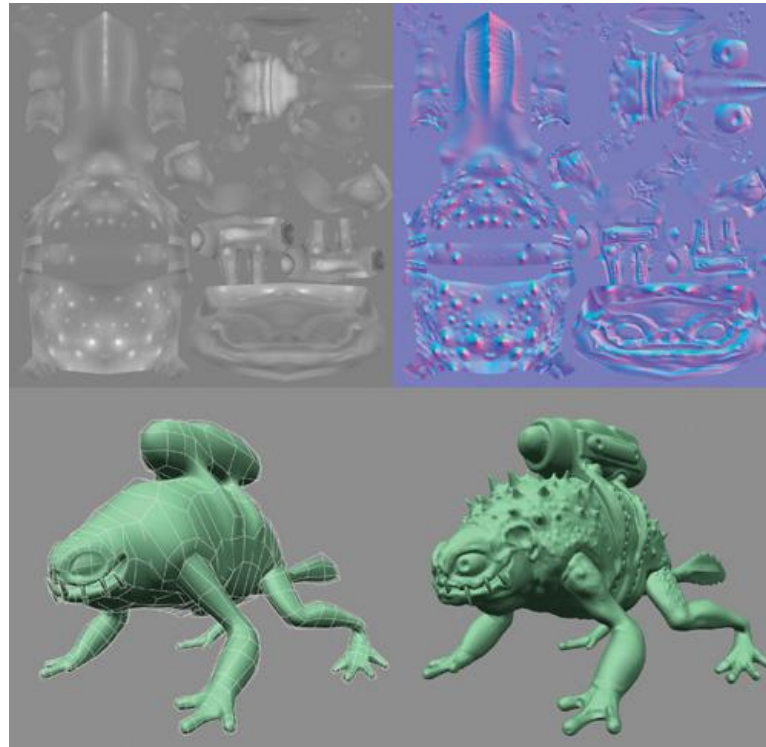
- **Tessellation shaders**

- Vertices transformed in vertex shader are assembled in the primitive assembler, and can be tessellated when a tessellation program is defined.
- Tessellation shader subdivides/tessellates triangles for higher geometric details.
- Unlike other shaders, tessellation shaders are composed of three stages.
 - Tessellation control shader (TCS): configure tessellation parameters
 - Tessellation primitive generation: perform tessellation
 - Tessellation evaluation shader (TES): perform post-transform new vertices
- Only, TCS and TES are programmable.

Tessellation Shaders

- **Typical usage : displacement mapping**

- Vertex buffer is defined only for simple control meshes.
- Geometric details are stored in height field or bump map, and tessellation shader adaptively elevates new vertices by the height field.



Adaptive Tessellation of Subdivision Surfaces with Displacement Mapping, GPU Gems 2, Ch. 7.

Geometry Shaders

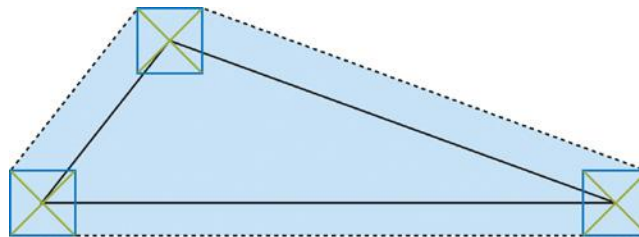
- **Geometry shaders**

- After primitive assembly or tessellation shaders, geometry can be processed in batch, if a geometry shader is defined.
- The geometry shader can clone or discard input triangles.
- The geometry shader is one of the worst-performance shaders. Unless we must use this, avoid using the geometry shader.

Geometry Shaders

- **Typical usages:**

- Manual culling:
 - Under a certain condition, discard triangles
- Layer redirection
 - Geometry shader can define multiple render targets with their own depth buffers.
- Enforcing per-face normal
 - We can generate per-face normal on the fly to replace per-vertex normal.
- Conservative rasterization
 - Expand a single triangle to multiple triangles to make its conservative bound.



Conservative Rasterization, GPU Gems 2, Ch. 42.

Transform Feedback

- **Transform feedback**

- Called the StreamOut in Direct3D 10/11/12
- Processed vertices can be stored in a buffer without passing them to fragment shader.

- **Typical usages:**

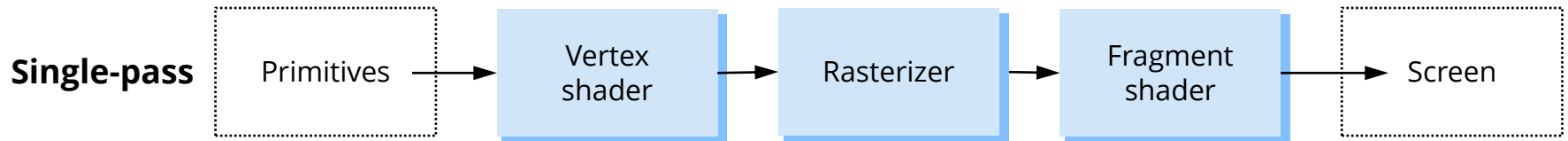
- Cache of tessellation and geometry shaders
 - store the results of tessellation or geometry shaders into the buffer and reuse them later without tessellation

Deferred Rendering Pipeline

Forward Rendering Revisited

- **Drawbacks of forward shading**

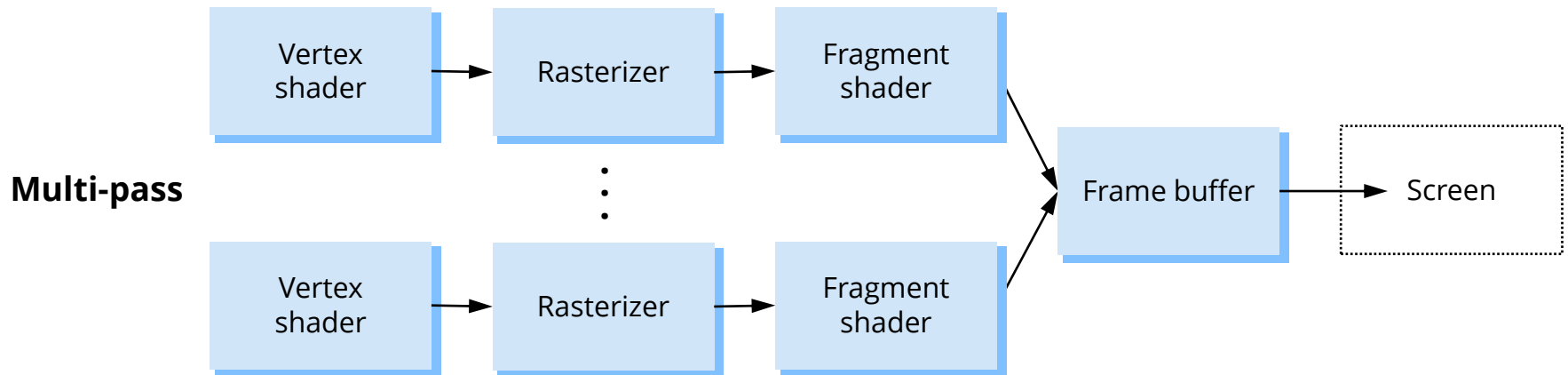
- Shading to present **natural (many) lights** is too expensive.
 - 10s to 10,000s, but 2-10 lights affecting a given pixel
- In single-pass rendering, all lights should be processed and rendered for each object in a single shader.
 - Complexity: $O(N_{\text{lights}} \times N_{\text{objects}})$
 - Invisible occluded fragments should also go through the costly shading.



Forward Rendering Revisited

- **Drawbacks of forward shading**

- In a multi-pass rendering, lights should be processed for affected objects iteratively.



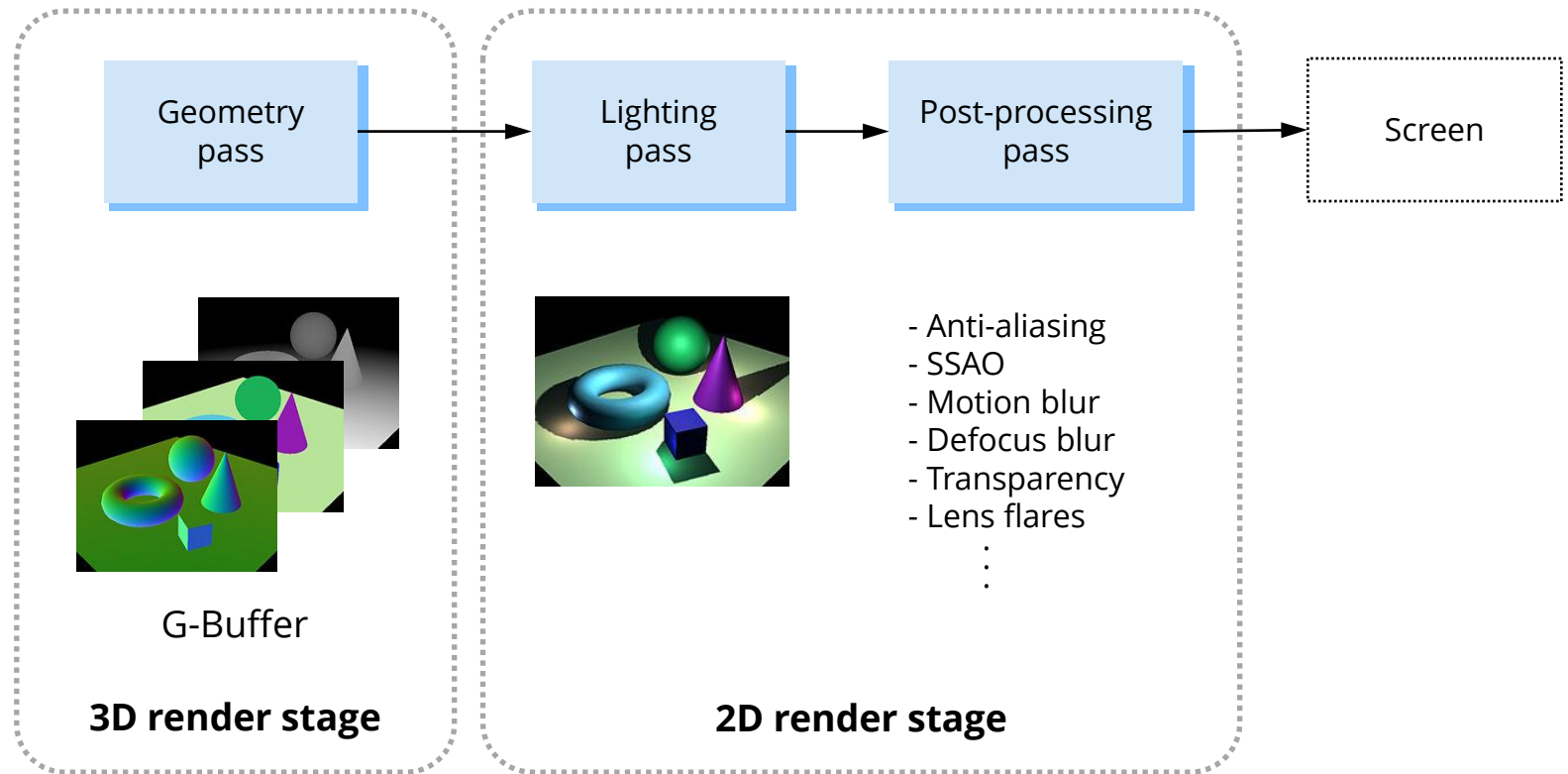
Deferred Rendering

- **General idea**

- Multi-pass approach yet with a single geometry pass
- Render attributes of the geometry and its material first on geometry pass.
 - The buffer is called the *G-buffer*.
 - Common attributes: position, normal, diffuse color, specular
- Perform shading later after the geometry pass
- Apply postprocessing to the lit image with additional attributes
 - Depth, motion vectors are common

Deferred Rendering

• Overview



Deferred Rendering

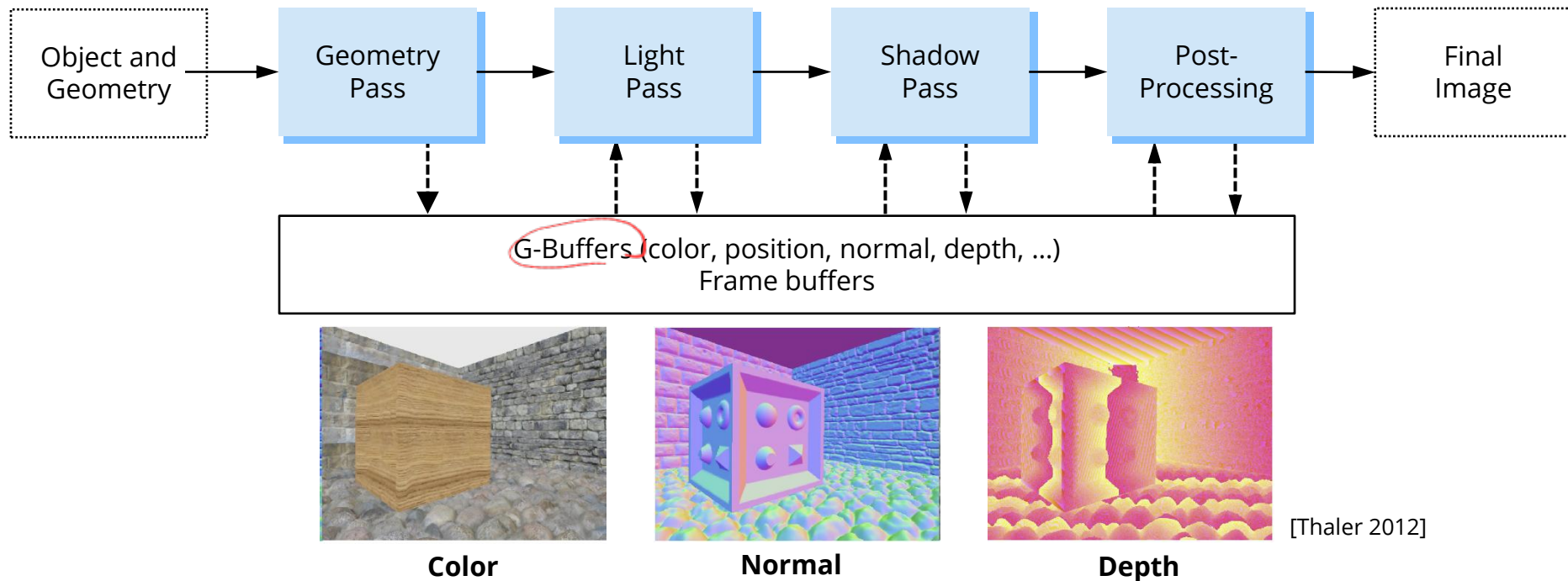
- **Commercial engines using deferred shading**
 - Nearly all the major games engines



Geometry Pass

- **Attributes to render to G-buffer in geometry pass**

- Common four attributes:
 - Normal, depth, diffuse color (albedo), specular color/shininess



Geometry Pass

- **Multiple render targets (MRTs) for G-buffers**

- G-buffers storing all the attributes need more than the maximum capacity of a single buffer (typically 128 bits, 4 channels)
- MRTs allow us to simultaneously render the attributes in a single pass
 - Up to 8 back buffers in general.
- Nonetheless, the amount of channels need to be reduced.
 - Bitwise encoding/packing are required
 - $4 \text{ channels} \times 8 \text{ bits} = 32 \text{ bits / render target}$

Geometry Pass

- **Example of the packed data structure for G-buffers**

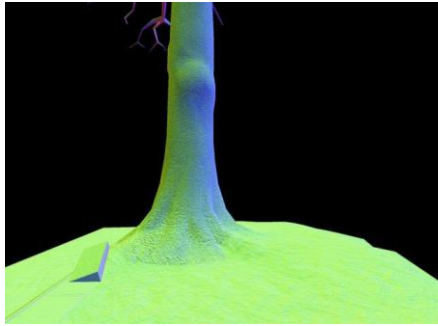
	R8	G8	B8	A8
DS		Depth 24bpp		Stencil
RT0	Lighting accumulation RGB8			Intensity
RT1	Normal.x FP16		Normal.y FP16	
RT2	Motion vectors XY RG8		Specular.power	Specular.intensity
RT3		Diffuse albedo RGB8		Sun.occlusion

The Rendering Technology of Killzone 2

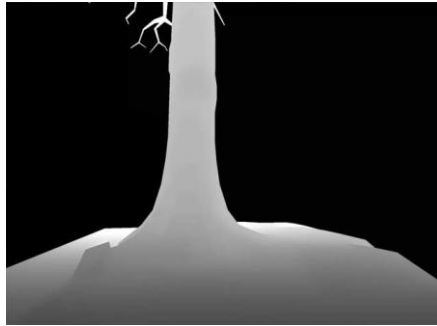
- Z of normal vector can be calculated from X and Y .
- Limitations: low precisions and overhead of packing/unpacking
- Yet, cheaper than bandwidth overhead

Examples

Deferred Shading, Shawn Hargreaves, GDC 2004



normals



depth



specular intensity



specular power



Diffuse color (albedo)



Deferred lighting result